SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Narappagari Sravya	**Reg. No.:**	192472192
Section / Batch:	B.E CSE AI / 2024	**Date:**	08-08-2026

Code of Conduct

*I **Narappagari Sravya** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: sravya

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

S → NP VP
NP → Det N VP
→ V NP

Det → the | a
N → student | teacher | book
V → reads | likes

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree a
teacher likes the book	Valid Parse Tree student
reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree reads
the student book	Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

he → singular, third person she → singular, third person it → singular, third person they → plural

runs → singular verb

writes → singular verb

run → plural verb

write → plural verb

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb): pass
```

Test Cases

Subject	Verb	Expected
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

S →	NP VP	[1.0]
NP →	John	[0.6]
NP →	Mary	[0.4]
VP →	runs	[0.5]
VP →	walks	[0.5]

Probability Formula

P(Sentence)

=

P(S→NP VP)

× P(NP)

× P(VP)

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

defsentence_probability(sentence): pass

Test Cases

Input Sentence Expected Probability

John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Experiment 1: CFG Rule Validation and Parse Tree Construction

CODE:

```
def parse_cfg(sentence):
    words = sentence.lower().split()
    # Grammar rules
    determiners = ["the", "a"]
    nouns = ["student", "teacher", "book"]
    verbs = ["reads", "likes"]
    # Sentence must contain exactly 5 words
    # S -> NP VP
    # NP -> Det N
    # VP -> V NP
    if len(words) != 5:
        return "Invalid Sentence"
    det1, noun1, verb, det2, noun2 = words
    # Check grammar
    if det1 not in determiners:
        return "Invalid Sentence"
    if noun1 not in nouns:
        return "Invalid Sentence"
    if verb not in verbs:
        return "Invalid Sentence"
```

```
    if det2 not in determiners:
        return "Invalid Sentence"
    if noun2 not in nouns:
        return "Invalid Sentence"
    # Parse tree
    tree = f"""
S
├── NP
│   ├── Det → {det1}
│   └── N → {noun1}
└── VP
    ├── V → {verb}
    └── NP
        ├── Det → {det2}
        └── N → {noun2}
"""
    return tree
# Main program
sentence = input("Enter a sentence: ")
result = parse_cfg(sentence)
if result == "Invalid Sentence":
    print(result)
else:
```

```
    print("Valid Parse Tree:")

    print(result)
```

Sample Output

Input:

the student reads a book

Output:

Valid Parse Tree:

S

├── NP

│ ├── Det → the

│ └── N → student

└── VP

  ├── V → reads

  └── NP

    ├── Det → a

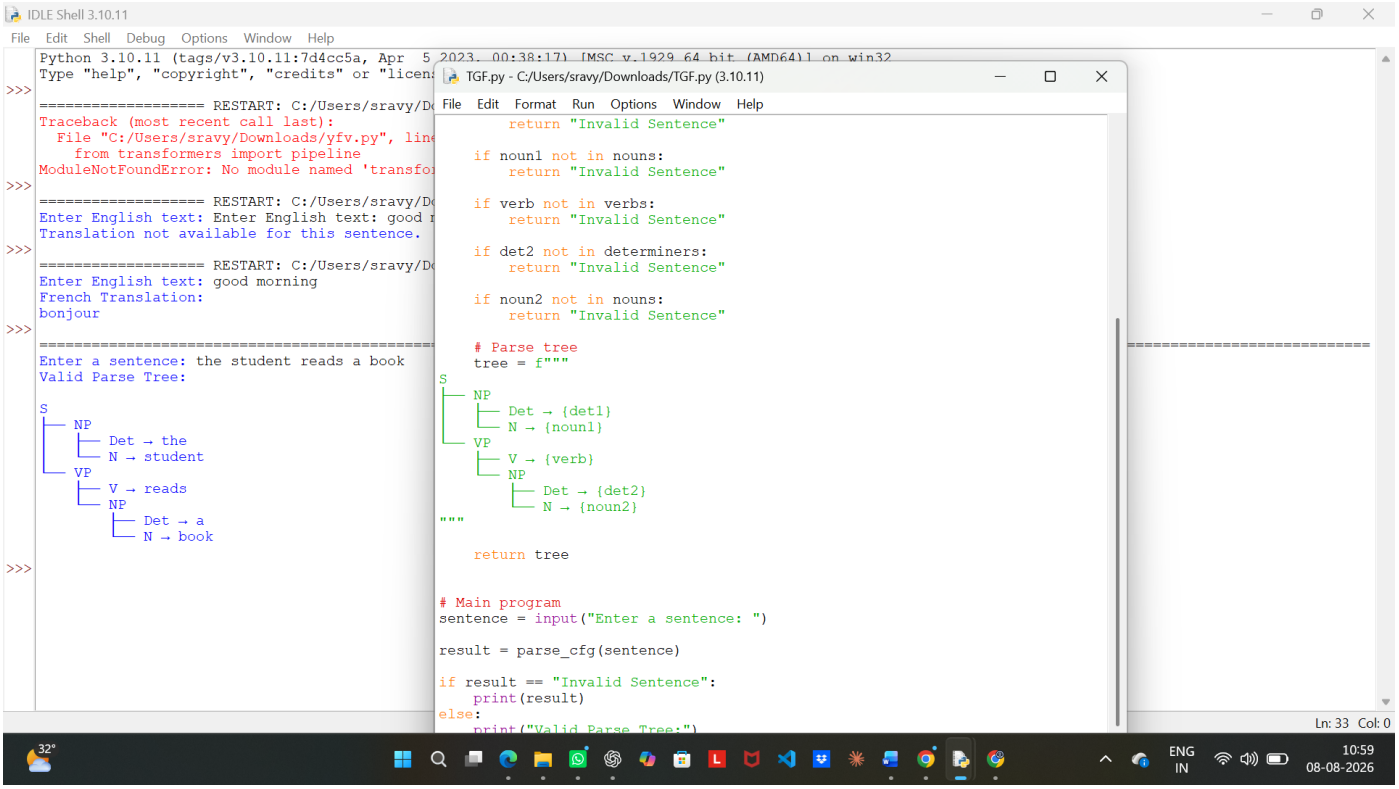
    └── N → book

Input:

student reads book

Output:

Invalid Sentence



Experiment 2: Agreement and Feature Structure Checking

CODE:

```
def check_agreement(subject, verb):
    # Feature structures
    subjects = {
        "he": {"number": "singular", "person": "third"},
        "she": {"number": "singular", "person": "third"},
        "it": {"number": "singular", "person": "third"},
        "they": {"number": "plural", "person": "third"}
    }
    verbs = {
        "runs": {"number": "singular"},
```

```
        "writes": {"number": "singular"},
        "run": {"number": "plural"},
        "write": {"number": "plural"}
    }
    # Check whether subject and verb exist
    if subject not in subjects or verb not in verbs:
        return False
    # Compare number features
    if subjects[subject]["number"] == verbs[verb]["number"]:
        return True
    return False
# Main program
subject = input("Enter subject: ").lower()
verb = input("Enter verb: ").lower()
result = check_agreement(subject, verb)
print("Agreement:", result)
```

Test Cases

Subject	**Verb**	**Output**
he	runs	True
he	run	False
they	run	True
they	runs	False
she	writes	True
she	write	False

Sample Output

Enter subject: he

Enter verb: runs

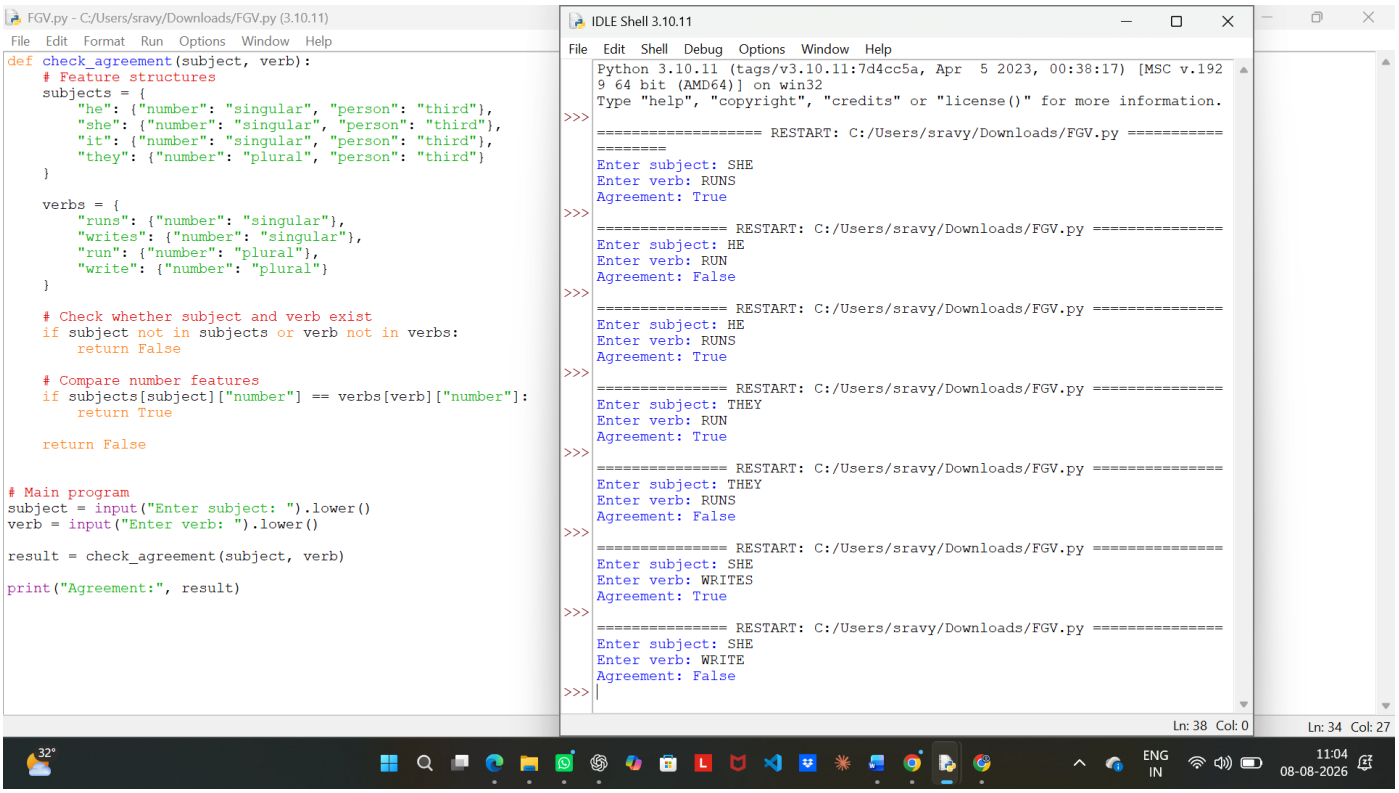
Agreement: True

Another example:

Enter subject: they

Enter verb: writes

Agreement: False



Experiment 3: PCFG Parsing

```
def sentence_probability(sentence):
    words = sentence.split()
    # Sentence must contain exactly two words
    if len(words) != 2:
        return 0.0
    subject = words[0]
    verb = words[1]
    # PCFG probabilities
    np_probability = {
        "John": 0.6,
        "Mary": 0.4
    }
    vp_probability = {
        "runs": 0.5,
        "walks": 0.5
    }
    # Check whether sentence can be generated
    if subject not in np_probability:
        return 0.0
    if verb not in vp_probability:
        return 0.0
```

```
    # S -> NP VP [1.0]
    s_probability = 1.0
    # P(Sentence) = P(S) × P(NP) × P(VP)
    probability = (
        s_probability
        * np_probability[subject]
        * vp_probability[verb]
    )
    return probability
# Main program
sentence = input("Enter a two-word sentence: ")
probability = sentence_probability(sentence)
print("Sentence Probability:", probability)
```

Test Cases

Input	**Probability**
John runs	0.3
John walks	0.3
Mary runs	0.2
Mary walks	0.2
Peter runs	0.0

Sample Output

```
Enter a two-word sentence: John runs
Sentence Probability: 0.3
```

For an invalid sentence:

Enter a two-word sentence: Peter runs

Sentence Probability: 0.0